

Machine Learning Final Assignment

Stephen Haskins: 21365534

January 5, 2026

Introduction

This assignment explores the repurposing of GPT transformer models to solve mathematical and boolean tasks. Each of the two sections involve building appropriate datasets, defining evaluation metrics, adapting model architectures, and analysing results across various arithmetic operations.

Part 1: Math GPT

1.1: Build an appropriate dataset

Designing an appropriate dataset is crucial for the performance of the Math GPT model. The dataset was generated via a python script that deterministically creates unique arithmetic problems and their respective solutions. This ensures all ground truth labels are completely accurate. The dataset includes a variety of operations such as addition, subtraction, multiplication, and division, with two levels of complexity selected via the ‘-basic’ and ‘-advanced’ flags.

Basic Expressions

The -basic flag generates expressions involving single-digit operands and simple operator combinations. This mode was used during early experimentation to validate that the transformer model, tokenisation method, and evaluation scripts were all functioning as expected.

Advanced Expressions

The -advanced flag extends this setup to more challenging two-digit arithmetic, including larger operand ranges and higher-magnitude outputs. This mode is intended to test the model’s ability to generalise to more complex mathematical tasks, and is used for final evaluation.

Basic examples	Advanced examples
$16 \times 31 = 496$	$52 \times 40 + 2 = 2082$
$77 - 50 = 27$	$5 - 9 \times 3 = -22$
$98 + 44 = 142$	$(1 \times 1) + 1 = 2$
$43 + 98 = 141$	$4 + (9 \times 2) = 22$
$93 - 48 = 45$	$65 + (4 \times 26) = 169$

Table 1: Example outputs for basic and advanced mathematical datasets

The dataset is then split into training and test sets using holdout validation. This ensures that evaluation is performed on unseen expressions, and provides a fair and reliable measure of model performance. Note, one pitfall of this design is that the division expression is under-represented. This is due to generating integer-only results, which significantly reduces the number of valid division expressions that can be generated. This constraint was added to avoid decimal results, which would add significant and disproportionate complexity to the division operation.

1.2: Define appropriate evaluation metrics

To evaluate the performance of the Math GPT model, multiple metrics were recorded to capture different aspects of the model’s performance. The primary metric used was accuracy, defined as

the percentage of correctly predicted expressions in the test set. This is the most task-appropriate metric, as the goal is to have the model produce exact matches to the ground truth solutions.

To provide further granularity and localisation to this metric, operation-wise accuracy was also recorded. This breaks down the overall accuracy by arithmetic operation (addition, subtraction, multiplication, division), revealing whether performance is uniform across operations, or whether the model finds certain operations more challenging.

The Character Error Rate (CER) was implemented retrospectively as results showed predictions were often off by just one or two digits. This metric captures how many single-character edits are required to transform the model output into the correct answer, meaning: CER focuses on near-miss errors, such as predicting 1025 instead of 1024, as opposed to complete hallucinations.

Finally, the Mean Absolute Error (MAE) measures the average absolute difference between the predicted and true numeric values. This provides insight to how far incorrect predictions deviate from the ground truth.

Using the above string and numerical based metrics in conjunction, provides a balanced and reliable evaluation framework for this assignment.

1.3: Explore adaptations of the model architecture for this application

1.4: Explore results for a range of arithmetic options

Addition

Addition is the best-learned operation, achieving an accuracy of 83.94%. This strong performance could be due to the relatively local and regular structure of addition. Multi-digit addition can commonly be decomposed into digit-wise operations with short dependency chains, where carries typically affect only adjacent digits. For example, expressions such as $12+13=25$ are frequently solved correctly. However, as the complexity of the addition increases, such as in $98+76=174$, the model struggles more, likely due to the longer dependency chains and carry operations involved.

Subtraction

Subtraction achieves the second-highest accuracy at 77.98%. This slightly weaker performance is expected, given the added complexity of borrowing compared to carrying, as well as sign handling for negative results. The model performs well on simpler cases such as $54-20=34$, but errors are more common when multiple borrow operations or negative results are involved, for example in expressions like $23-89$.

Division

Division achieves an accuracy of 62.26%, which is notably lower than addition and subtraction but still demonstrates non-trivial learning. As mentioned, division is under-represented in the dataset, with only 53 division examples appearing in the test set, compared to roughly one thousand examples for each of the other operators. Typical correctly solved cases include expressions such as $48/6=8$ or $81/9=9$. Given the small sample size, this accuracy estimate is noisier and less reliable, and performance would be expected to degrade sharply if non-integer quotients or decimal representations were introduced.

Multiplication

Multiplication is the weakest operation by a large margin, with an accuracy of just 13.51%. This behaviour is expected and highlights a fundamental limitation of the model. Multi-digit multiplication requires quadratic interactions between digits, meaning every digit in operand A

interacts with every digit in operand B. In addition, long dependency chains due to multiple carries, and the generation of comparatively long outputs complicates this operation significantly. For example, in cases such as $53 \times 71 = 3763$, the model must correctly generate multiple digits in sequence, and a single incorrect carry can corrupt the entire result. Results show, the model often produces outputs that are numerically incorrect but with near-miss predictions that differ by one or two digits. For this reason, the CER metric was used.

CER and MAE

The character error rate of 0.2290 indicates that, on average, roughly 23% of characters in the model's predicted answers differ from the ground-truth output. This reflects errors at the character level rather than purely numeric correctness. Multiplication is expected to dominate this metric, as its longer outputs make it especially sensitive to cascading mistakes. This would be the first expansion point for future work, creating more detailed analysis on CER, such as per-operation, and reducing character-level errors to improve overall accuracy.

The mean absolute error of 116.94 highlights a similar asymmetry across operations. Errors in addition and subtraction typically produce small numeric deviations, whereas multiplication errors often lead to large numerical discrepancies. A single incorrect digit in a higher place value can result in errors of hundreds or more, causing multiplication to disproportionately inflate MAE despite being only one category of operation.

Part 2: Boolean GPT

2.1: Build an appropriate dataset

Similar to the math model, the Boolean GPT model's dataset was generated deterministically via a python script. This script creates unique boolean expressions and their respective solutions, ensuring ground truth labels are completely accurate. The dataset includes boolean logical operations such as AND, OR, NOT, and XOR, with two levels of complexity selected via the 'basic' and 'advanced' flags. Similar to before, the basic dataset was just used to confirm the model and evaluation scripts were functional, while the advanced dataset was used for final evaluation.

Issues with dataset

Due to the nature of Boolean logic, the number of unique expressions is much smaller than in the previous math arithmetic. There are only two operands, True and False, and a limited set of operators (AND, OR, XOR, NOT), which means that all can be exhaustively enumerated with relatively few samples. As a result, the dataset contains only a few hundred unique expressions.

Increasing the dataset size would therefore only possible by scaling the complexity of expressions.

Finally, because the model is character-level, accuracy is sequence-based rather than label-based. The model must generate the exact target string character-by-character after seeing the prompt up to the equals sign, so even a logically correct intention that produces an incorrect sequence (e.g., wrong spelling, extra whitespace, or partial tokens) is treated as incorrect. This is overcome in the following adaptations section.

2.2: Define appropriate evaluation metrics

Unlike math arithmetic, boolean logic has either a True or False output, meaning this task can be treated as a binary classification problem.

The primary metric is accuracy, defined as the proportion of test expressions for which the model’s generated output matches the ground-truth Boolean value. This is the most appropriate headline metric for Boolean logic, as any deviation from the correct value represents a complete logical failure rather than a partial error. Because the output space is binary and unambiguous, exact match accuracy directly measures task success

To gain more detailed insight into model behaviour, operator-wise accuracy is also reported. This metric breaks exact match accuracy down by the dominant logical operator appearing in each expression (AND, OR, XOR, NOT). This analysis is particularly important for Boolean logic, as different operators have fundamentally different properties. In particular, XOR is not linearly separable and is known to be more difficult for simple models to learn. Operator-wise accuracy therefore helps identify whether the model has learned general logical reasoning or is relying on shallow heuristics that work only for certain operators.

In addition, standard classification metrics derived from the confusion matrix are computed: precision, recall, and F1 score. These metrics complement accuracy by exposing potential biases in the model’s predictions. For example, a model that defaults to predicting False on difficult expressions could achieve deceptively high accuracy if the dataset is balanced, but would exhibit poor recall for the True class. Precision, recall, and F1 score make such failure modes explicit and ensure that performance is not being overstated by a single aggregate metric.

Finally, to provide further granularity to the accuracy metric, operation-wise accuracy was also recorded. This provides the accuracy values for expressions with either one or two operators.

Together, these metrics provide a rigorous and interpretable evaluation framework for assessing the Boolean GPT model’s performance.

2.3: Explore adaptations of the model architecture for this application

talk about dataset and character level output - how to adapt the model to this task

2.4: Explore results for a range of arithmetic options

This section evaluates the Boolean GPT model’s ability to learn and apply different logical operators, using quantitative results and representative examples. Evaluation on 49 unseen test expressions yields an overall exact match accuracy of 75.51%, with a validity rate of 100%, indicating that the model consistently generates well-formed Boolean outputs despite operating at the character level.

Operator-wise results show clear variation in difficulty. OR is learned perfectly, achieving 100% accuracy, with expressions such as $1 \mid 0=1$ and $(0 \mid 0) \mid 0=0$ always evaluated correctly. This reflects the simplicity and monotonic structure of OR, which is easy for the model to capture even in chained expressions. AND also performs well, with an accuracy of 76.92%, and is generally correct in cases like $1 \& 0 \& 0=0$. Errors typically arise when AND is combined with other operators, indicating challenges with composition rather than misunderstanding AND itself.

NOT is also solved perfectly in this evaluation, but it is under-represented, appearing in only a single test case. While simple negation such as $! 0=1$ is handled correctly, this result should be interpreted cautiously due to the small sample size.

XOR is the weakest operator, with an accuracy of 68.97%. Many errors involve chained or nested XOR expressions, such as $1 \wedge 1 \wedge 1$ or $0 \wedge (1 \wedge 0)$, reflecting the non-linearly separable nature of XOR and the need to track parity across operands. This makes XOR substantially harder than AND or OR for the model.

Expression complexity further explains these results. All single-operator expressions are solved correctly, while accuracy drops to 73.33% for expressions containing two operators. This indicates

that the main failure mode is compositional reasoning, particularly when multiple operators interact.

The character error rate of 0.2449 reflects incorrect final truth values rather than malformed outputs, as Boolean outputs are short. Precision, recall, and F1 score are all 0.7391, indicating balanced performance across True and False classes. Overall, the model reliably learns simple Boolean operators, but struggles with XOR and with deeper compositional structure, highlighting the limits of character-level reasoning for non-linear logical operations.